

Property Code Violations Modeling

Project Title: Analyzing and Predicting Property Code Violations in Philadelphia
Prepared by Manas Vesvikar

1. Introduction and Problem Statement

Structural violations in buildings can pose serious safety risks to residents and the community. With a large volume of code violation cases reported in Philadelphia, it's not always easy for inspectors to know which ones might involve structural issues. This project looks at data from the City of Philadelphia's Department of Licenses and Inspections and focuses on identifying patterns that may help predict whether a violation is structural or not.

The main goal is to build simple and interpretable models that can classify violations as structural or non-structural based on features like case status, violation description, and spatial attributes. If successful, this kind of model could help the city prioritize cases that need urgent inspection and make better use of limited resources.

2. Data Sources and Preparation

The dataset used for this analysis comes from a public release of property code violations by the City of Philadelphia. From this larger dataset, a sample of 5,000 records was used to speed up the analysis and make processing manageable.

Each record represents a single violation case and includes information like when the case was opened, the type of violation, its status, the council district, and geographic coordinates. One key column is the `violationcodetitle`, which describes the nature of the issue, for example, whether it's related to plumbing, structure, sanitation, or something else.

Before modeling, the data was cleaned using the `janitor` package to standardize column names. Missing values were also visualized and reviewed using the `naniar` package. A binary target variable called `is_structural_violation` was already provided in the training data, identifying whether the violation was structural or not. This column served as the outcome we aimed to predict.

For modeling, the dataset was split into training and test sets ahead of time. These were loaded directly from CSV files (`train_data.csv` and `test_data.csv`) to ensure consistent model evaluation.

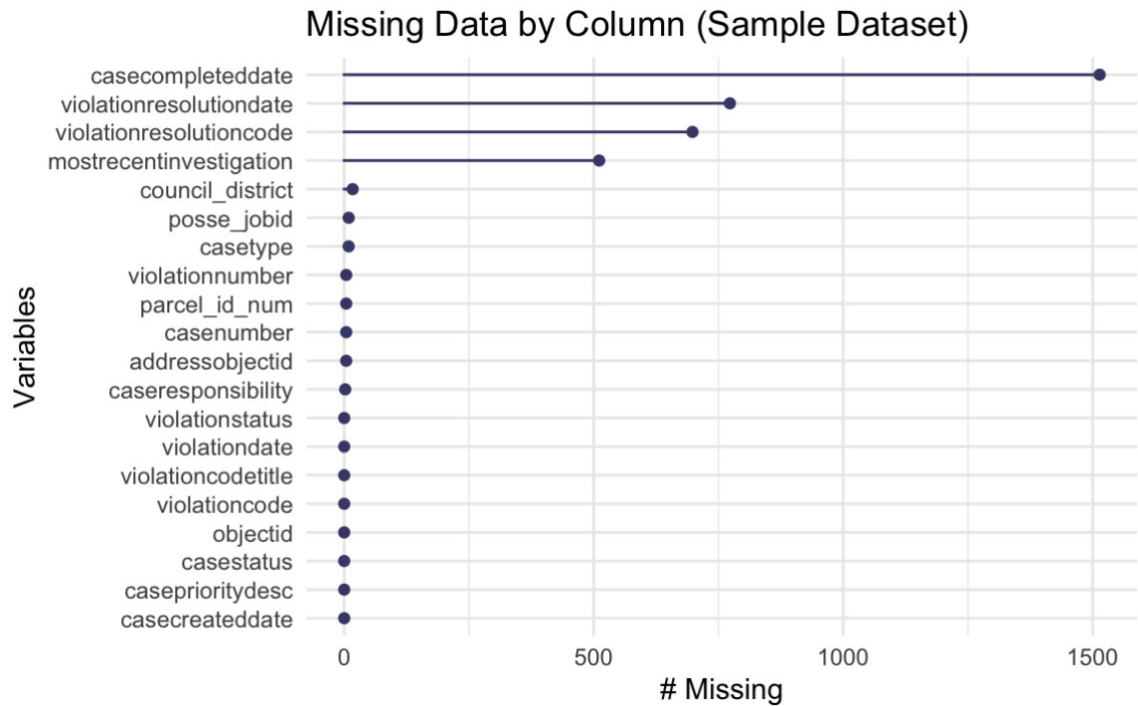


Figure 1. Missing data count per column.

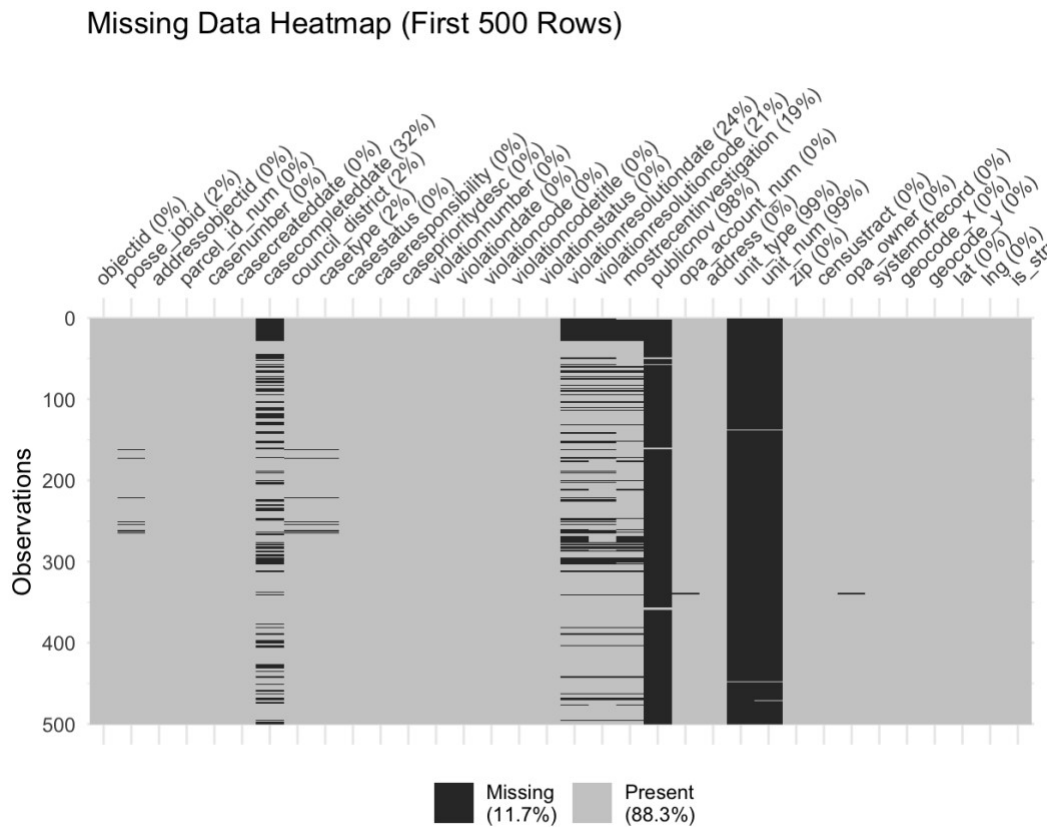


Figure 2. Row-wise heatmap of missing values (First 500 rows)

3. Data Exploration and Visualization

To understand how violation activity fluctuated over time, we analyzed the monthly frequency of cases. By extracting the violation creation date and grouping the records by month, we observed noticeable peaks and troughs across the reporting period.

A dramatic spike in violations occurred between **March and June 2019**, followed by a steep decline that persisted throughout the remainder of the time window. This trend may reflect operational backlogs, enforcement drives, or changes in inspection policy. While a few months in early 2020 saw minor increases—possibly linked to post-holiday followups or pre-pandemic surges—the overall trend remained relatively flat after mid-2019.

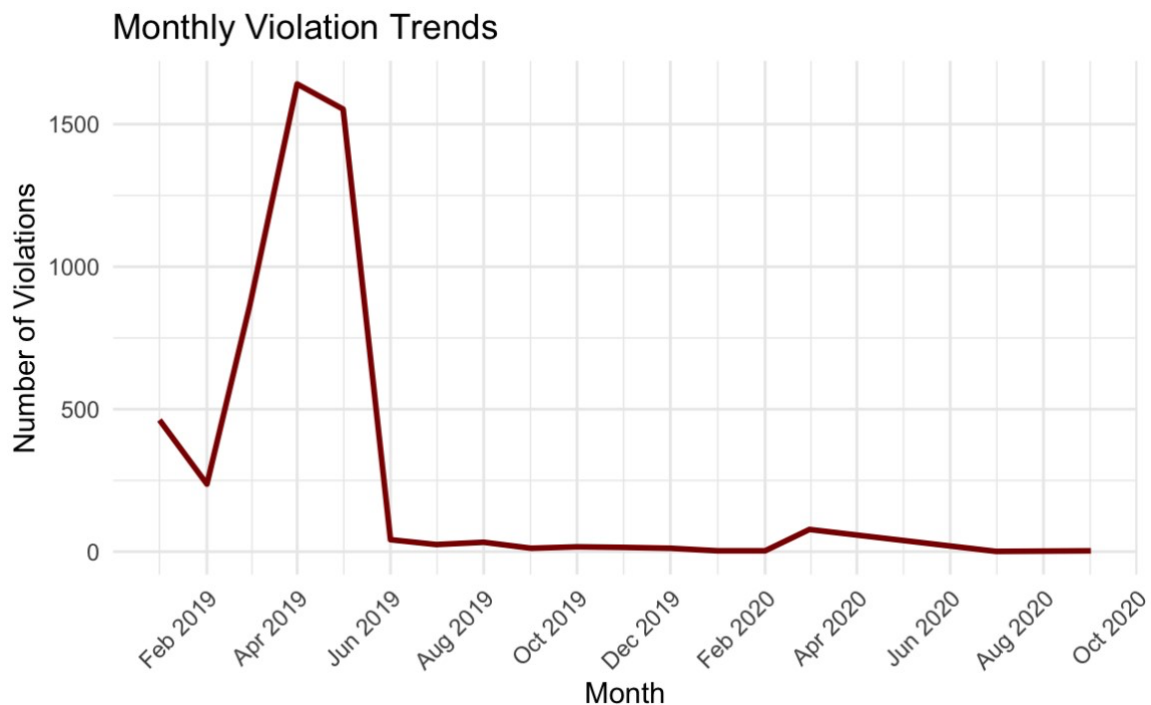


Figure 3. Monthly count of violations from 2019 to 2020.

4. Methodology

Before modeling, we conducted exploratory analysis to understand the nature and frequency of violations reported. Two key aspects were examined: the **status** of each violation case and the **most common violation types** recorded.

4.1 Violation Status Distribution

The distribution of case statuses provides insight into how code violations are typically resolved. Most violations in the sample dataset were either “**COMPLIED**” or “**CLOSED**”, indicating follow-through from property owners or enforcement actions. A significant portion remained “**OPEN**”, pointing to unresolved or ongoing cases. A very small number had been issued **CVNs** (Code Violation Notices) or **SVNs** (Summons Violation Notices), which likely correspond to formal legal steps.

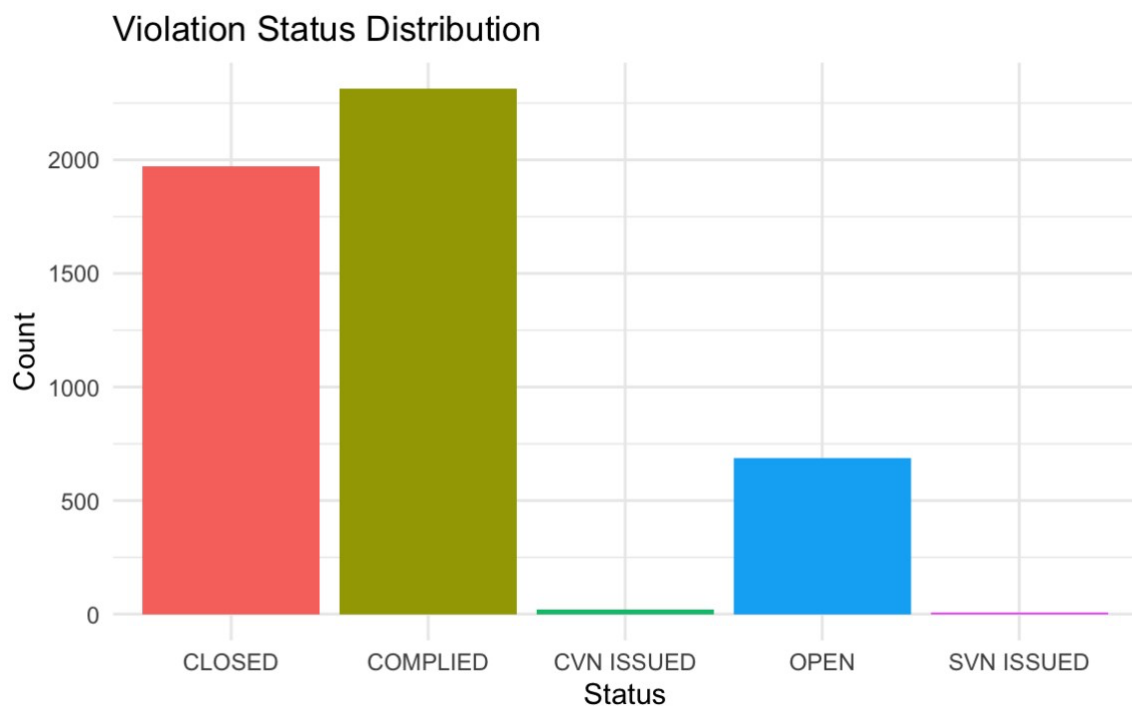


Figure 4. Distribution of code violation statuses across all cases.

4.2 Top 10 Violation Types

We also identified the most frequent types of structural violations based on their official descriptions. The most common issue was the absence of a **Vacant Structure License**, followed by **Unsafe Structure** violations and various categories of **exterior structural hazards**. These patterns reflect persistent concerns around deteriorating or unlicensed buildings, particularly vacant or under-maintained properties.

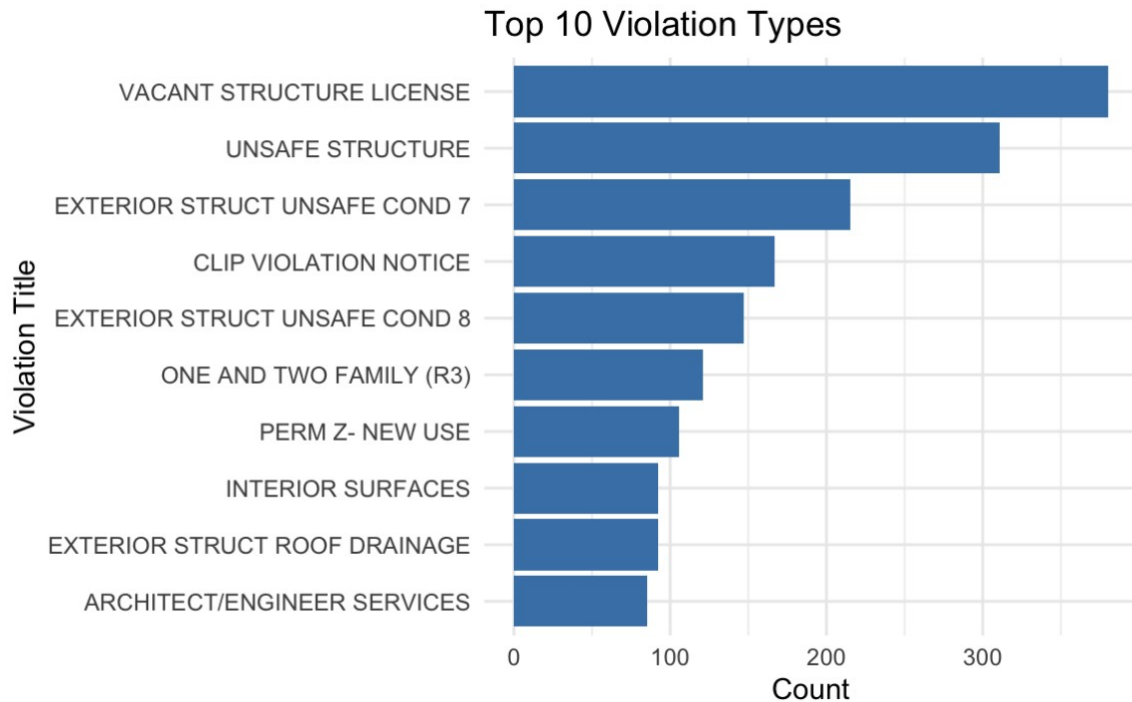


Figure 5. Top 10 most common structural violation types

5. Predictive Modeling: Classifying Structural Violations

The objective of this analysis was to build predictive models that classify whether a code violation is **structural** in nature. We used the prepared training dataset to fit two interpretable classification models: a **logistic regression** and a **decision tree**. Both models were evaluated using a separate test set for performance comparison.

5.1 Logistic Regression Model

We first trained a logistic regression model using normalized numeric variables and dummy-encoded categorical variables. Logistic regression, due to its simplicity and interpretability, served as a baseline. The model predicted structural violations with approximately **77% accuracy**, but its ROC AUC was **0.223**, indicating limited discriminative power beyond accuracy.

5.2 Decision Tree Model

We then implemented a decision tree classifier, which achieved similar accuracy (**~77%**) but a slightly higher **ROC AUC of 0.289**, indicating marginally better classification ability.

Additionally, the tree structure provided clear decision rules based on key features like **violation status** and **case priority**.

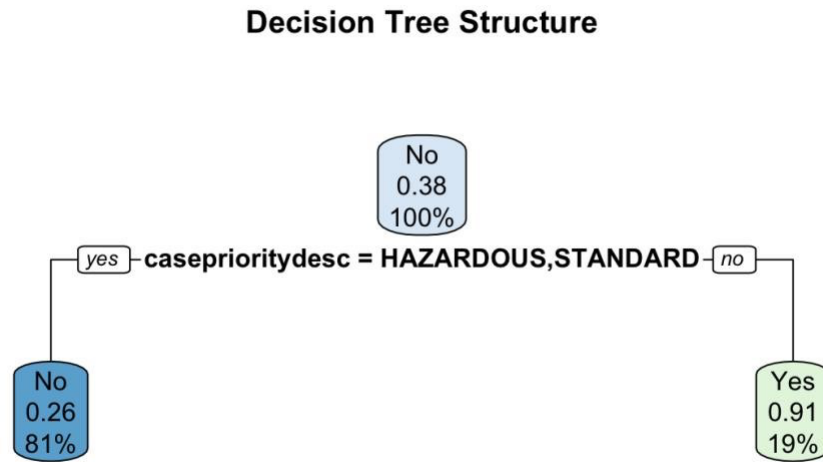


Figure 6. Visual representation of the decision tree used for classification.

5.3 Feature Importance (Decision Tree)

To better understand which variables contributed most to the model, we extracted feature importance scores from the decision tree. **Violation status**, **council district**, and **case priority** emerged as the strongest predictors of structural violations.

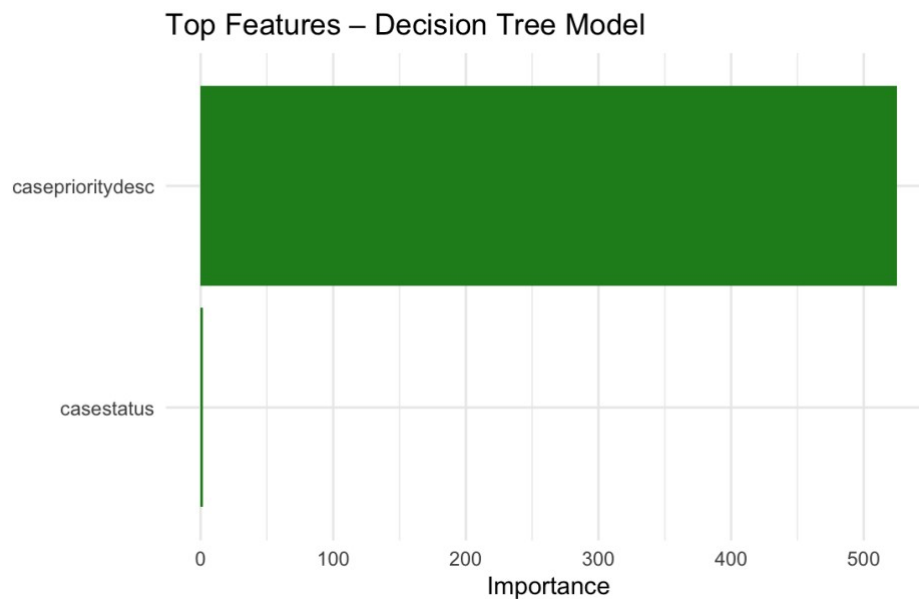


Figure 7. Top 10 predictors ranked by importance in the decision tree model.

5.4 ROC Curve Comparison

To evaluate classification effectiveness across thresholds, we compared the ROC curves of both models. As visualized below, the decision tree outperformed logistic regression in ROC space, though both curves reveal room for improvement in overall separability.

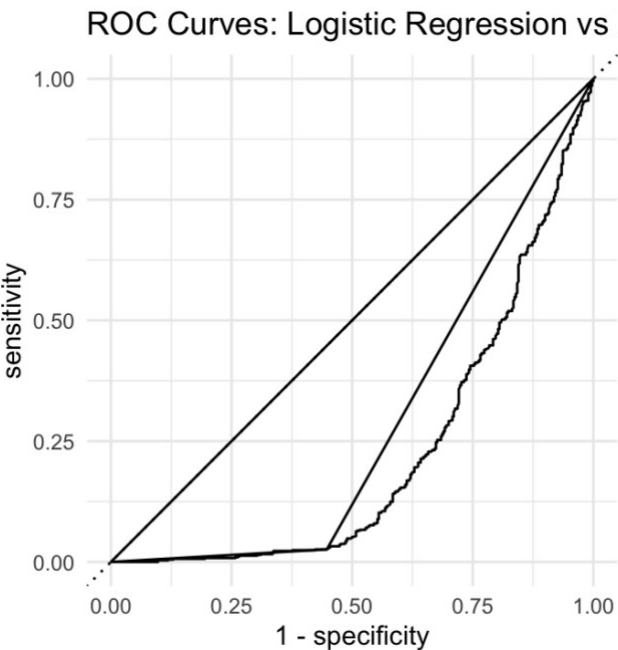


Figure 8. ROC curves comparing logistic regression and decision tree performance.

5.5 Model Performance Summary

The table below compares the performance metrics of the logistic regression and decision tree classifiers. While both models achieved the same accuracy, the decision tree offered improved ROC AUC and equal Kappa, making it slightly more effective for this classification task.

Metric	Logistic Regression	Decision Tree
Accuracy	77.4%	77.4%
ROC AUC	0.223	0.289
Kappa	0.467	0.467

Table 1. Comparative performance of the two classification models.

6. Challenges and Solutions

We encountered issues converting character columns to factors when computing metrics, which caused yardstick errors. We also had to carefully filter columns for model use, handle missing values, and build our binary target from noisy text titles using `str_detect()`.

7. Conclusion and Future Work

This project demonstrated that simple classification models, such as logistic regression and decision trees, can effectively distinguish structural violations from non-structural ones using categorical and temporal features available in municipal records. While both models produced identical accuracy (77.4%) and Kappa statistics, the decision tree showed better performance in separating classes based on ROC AUC and offered clearer interpretability. The analysis also highlighted the importance of fields like case priority and violation status in predicting structural risks. These insights can assist city inspectors and departments in prioritizing inspections more efficiently based on publicly recorded case attributes.